

✓ 30 DOMANDE TEORICHE (con risposte sintetiche)

JAVA BASE (1–6)

1) Differenza tra tipo primitivo e wrapper

I primitivi (`int`, `double`) non sono oggetti, i wrapper (`Integer`, `Double`) sono classi e possono essere usati nelle collezioni.

2) Differenza tra `==` e `equals()`

`==` confronta i riferimenti (memoria), `equals()` confronta il contenuto (se implementato correttamente).

3) Cos'è l'overloading

È la definizione di più metodi con lo stesso nome ma parametri diversi (numero/tipo/ordine).

4) Perché `String` è immutabile

Per sicurezza e prestazioni (string pool). Una modifica crea una nuova stringa.

5) Differenza tra `StringBuilder` e `String`

`StringBuilder` è mutabile ed efficiente per concatenazioni ripetute.

6) Cos'è lo scope di una variabile

È la zona del codice in cui la variabile è visibile (es. dentro un metodo o un blocco).

OOP (7–14)

7) Cos'è l'incapsulamento

Nascondere i dati con `private` e controllare l'accesso tramite getter/setter.

8) Differenza tra classe astratta e interfaccia

Classe astratta: può avere stato e metodi concreti. Interfaccia: definisce un contratto (metodi astratti, default, static).

9) Cos'è il polimorfismo

Un riferimento di tipo padre può puntare a oggetti figli, chiamando il metodo override corretto.

10) Differenza tra override e overload

Override: ridefinizione in sottoclasse. Overload: stesso nome, parametri diversi.

11) A cosa serve `super`

Per richiamare costruttore o metodi della classe padre.

12) Cosa significa `static`

Un membro `static` appartiene alla classe, non all'oggetto.

13) Cosa significa `final`

Variabile: non modificabile. Metodo: non sovrascrivibile. Classe: non estendibile.

14) Cos'è il costruttore

Metodo speciale che inizializza l'oggetto, chiamato con `new`.

COLLECTIONS + GENERICS (15–20)

15) Differenza tra List e Set

List ammette duplicati e mantiene ordine. Set non ammette duplicati.

16) Differenza tra `ArrayList` e `LinkedList`

`ArrayList`: accesso veloce per indice. `LinkedList`: inserimenti/cancellazioni frequenti più efficienti.

17) Differenza tra `HashSet` e `TreeSet`

`HashSet`: non ordinato. `TreeSet`: ordinato (richiede `Comparable/Comparator`).

18) Differenza tra `HashMap` e `TreeMap`

`HashMap`: accesso veloce, non ordinato. `TreeMap`: ordinato per chiave.

19) Cos'è un Generic

È un tipo parametrico, es. `List<String>`, che evita cast e aumenta sicurezza.

20) Comparable e Comparator

Comparable: ordinamento naturale nella classe (`compareTo`). Comparator: ordinamento esterno personalizzato (`compare`).

ECCEZIONI (21–23)

21) Differenza tra checked e unchecked exception

Checked: obbligo di gestione (es. `IOException`). Unchecked: `RuntimeException`, non obbligo (es. `NullPointerException`).

22) A cosa serve `try-catch-finally`

Gestire errori e garantire esecuzione di codice finale (chiusura risorse).

23) Cosa fa `throw`

Lancia manualmente un'eccezione.

FILE (24–25)

24) Differenza tra file di testo e file binario

Testo: leggibile (CSV, TXT). Binario: non leggibile direttamente (immagini, oggetti serializzati).

25) `BufferedReader` e `FileReader`

`FileReader` legge caratteri, `BufferedReader` migliora efficienza e permette `readLine()`.

DATABASE + JDBC (26–28)

26) Cos'è JDBC

API Java per collegarsi a database ed eseguire query SQL.

27) Differenza tra `Statement` e `PreparedStatement`

`PreparedStatement` è più sicuro (anti SQL injection) e più efficiente per query ripetute.

28) Cos'è la SQL Injection

Attacco in cui l'utente inserisce SQL malevolo nei campi input alterando la query.

SPRING BOOT + REST + JPA (29–30)

29) Cos'è la Dependency Injection

Spring crea e gestisce oggetti (bean) e li “inietta” dove servono evitando `new`.

30) Cos'è una REST API

Servizio web basato su HTTP che espone risorse tramite metodi GET/POST/PUT/DELETE e risposte JSON.